



西南科技大学

**SOUTHWEST UNIVERSITY OF SCIENCE AND
TECHNOLOGY**

PROJECT TITLE

CloudPOS – Role-Based Point of Sale System

TEAM MEMBERS:

FRIMPONG MAAME ACHIAA (4420230903)

TRUONG QUANG NGHIA (4420230904)

KARIM OUSSAMA (4420230906)

DEGREE: BACHELOR OF ENGINEERING

COURSE INSTRUCTOR: Dr LORD BLESS

Table of Contents

CHAPTER ONE	4
1. INTRODUCTION	4
1.1. BACKGROUND	4
1.2. PROBLEM STATEMENT	4
1.3. OBJECTIVES	5
1.4. SCOPE	5
CHAPTER TWO	7
2.SYSTEM ANALYSIS AND REQUIREMENTS	7
2.1. SYSTEM OVERVIEW	7
2.2. FUNCTIONAL REQUIREMENTS	7
2.3. NON-FUNCTIONAL REQUIREMENTS	8
2.4. USE CASE DIAGRAM	9
FIGURE 1	9
2.4.1 EXPLANATION	9
CHAPTER THREE	11
3. SYSTEM AND SOFTWARE DESIGN.....	11
3.1. POS SYSTEM – CLOUDPOS - SYSTEM ARCHITECTURE & DESIGN PATTERNS.....	11
3.1.1. HIGH-LEVEL SYSTEM ARCHITECTURE	11
3.1.2. DESIGN PATTERNS USED.....	12
3.2. OBJECT-ORIENTED CLASS DIAGRAM.....	13
3.2.1. COMPLETE UML CLASS DIAGRAM.....	13
3.2.2. OFFLINE ARCHITECTURE	14
3.2.3. EXPLANATION	15
3.3. SEQUENCE DIAGRAM	16
3.3.1. KEY RELATIONSHIPS SUMMARY	17
3.4. ACTIVITY DIAGRAM	18
3.5. WORK BREAKDOWN STRUCTURE (WBS).....	19
3.6. GANTT CHART	22
3.7. ACTIVITY NETWORK DIAGRAM	23
3.7.1. CRITICAL PATH ANALYSIS (PERT/CPM)	24

CHAPTER FOUR25

4. IMPLEMENTATION.....25

4.1. TOOLS AND TECHNOLOGIES25

4.1.1. PROGRAMMING LANGUAGE.....25

4.1.2. GRAPHICAL USER INTERFACE (GUI) FRAMEWORK.....25

4.1.3. DATABASE TECHNOLOGIES26

4.1.4. CLOUD & SYNCHRONIZATION26

4.1.5. PDF GENERATION (RECEIPTS)27

4.1.6. DEVELOPMENT TOOLS27

4.1.7. HARDWARE REQUIREMENTS (DEPLOYMENT).....28

4.2. TECHNOLOGY STACK SUMMARY DIAGRAM.....29

4.3. SYSTEM ARCHITECTURE.....30

4.3.1. HIGH-LEVEL ARCHITECTURE OVERVIEW30

4.3.2. ARCHITECTURE DIAGRAM31

4.3.3. OFFLINE-FIRST SYNC ARCHITECTURE.....32

4.3.4. SECURITY ARCHITECTURE33

4.4. SYSTEM MODULES.....34

4.5. KEY FEATURES39

4.5.1. FEATURE SUMMARY TABLE39

4.6. SCREENSHOTS.....41

FIGURE 2. LOGIN PAGE41

FIGURE 3. ADMIN’S DASHBOARD41

FIGURE 4. PRODUCTS42

FIGURE 5. CASHIER’S DASHBOARD42

FIGURE 6. MANAGER’S DASHBOARD43

Chapter one

1. Introduction

1.1. Background

Retail businesses, especially small to medium-sized enterprises (SMEs), face significant operational challenges in managing over-the-counter sales, tracking staff actions, preventing revenue loss, and maintaining inventory accuracy. Traditional POS systems often lack structured shift management, granular access control, and reliable data synchronization when internet connectivity is intermittent.

The goal is to build a **desktop-based POS system** using Python, integrated with a cloud database, that enforces strict role-based access control (RBAC), supports shift-level accountability, and enables offline operation with cloud synchronization.

Key Technical Drivers

- **Cloud-First but Offline-Capable:** Centralized data management with local resilience.
- **Role-Based Access Control (Admin, Manager, Cashier):** Prevents unauthorized actions and reduces internal fraud.
- **Shift-Level Reconciliation:** Ens every cashier's session is tracked from opening to closing balance.

1.2. Problem Statement

Many retail POS systems in use today suffer from the following critical issues:

1. **Weak Access Control** – Cashiers and managers often share login credentials, leading to audit trails that cannot identify individual responsibility.
2. **No Formal Shift Management** – There is no systematic way to track opening cash balances, sales during a shift, and closing counts, making cash theft or errors hard to detect.
3. **Revenue Leakage** – Without split payment tracking, receipt generation, and approval workflows, revenue can be lost through unrecorded transactions or unauthorized discounts.
4. **Poor Offline Handling** – When internet fails, many systems stop working entirely, causing business disruption.

5. **Inflexible User Account Creation** – Public registration or insecure account creation allows unauthorized system access.

Therefore, there is a clear need for a secure, modular, offline-first POS system that enforces role hierarchy, shift accountability, and reliable transaction logging.

1.3. Objectives

1. Implement a secure authentication module using cloud authentication with hashed passwords and Row Level Security (RLS).
2. Enforce a 3-tier role hierarchy (Admin, Manager, Cashier) where only Admins can create new staff accounts.
3. Build a POS terminal with fast product search (by barcode or name), cart management, and split payment (cash + bank transfer).
4. Automatically generate PDF receipts for every completed sale (simulating thermal printer output).
5. Implement shift management: open shift (with cash input) and close shift (with counted cash and discrepancy calculation).
6. Support offline operation – continue sales when internet is lost, then sync to cloud when reconnected.

1.4. Scope

In Scope (What the Project Will Deliver)

1. **Fully functional desktop POS application** (Windows/Linux compatible) written in Python.

2. **Cloud database integration** with:

- Product table (barcode, name, price, attributes, image URL)
- Users table (hashed password, role, name, shift history)
- Sales transaction table (items, total, split payment details, shift ID, timestamp)
- Shift table (cashier ID, open time, close time, opening balance, closing balance, discrepancy)

3. **Authentication & role-based UI** – Different dashboards for Admin, Manager, Cashier.

4. **Shift operations** – Open/close shift with cash confirmation.

5. **Sales terminal** – Cart, payment, change calculation, split payment, PDF receipt.

6. **Product management (CRUD)** – Only for Admin and Manager, with validation.

7. **Offline-first sync mechanism** – Local queue + cloud upload on reconnect.

8. **Basic reporting** – Shift summary, sales by date.

Chapter two

2. System Analysis and Requirements

2.1. System Overview

The Point of Sale (POS) System is designed to manage retail sales operations efficiently, securely, and reliably. It allows cashiers to process customer purchases, accept payments, generate receipts, and manage daily register activities.

The system supports different user roles such as Admin, Manager, and Cashier, where each role has specific permissions based on responsibility. Cashiers handle daily transactions, managers supervise sales and cash activities, and admins manage users, roles, and system settings.

A key part of the system is its offline-first capability. Sales can continue even when there is no internet connection by storing data locally in SQLite. Once the connection is restored, the system synchronizes the local data with the central PostgreSQL database.

2.2. Functional Requirements

The system consists of the following core modules:

1. Module Authentication & Role Management

- Centralized Login: Uses Cloud Auth for user authentication.
- 3-Tier Role Hierarchy:

Admin: Full authority to manage staff accounts and access all system functions.

Manager: Manages products, approves sensitive tasks, and handles shift reconciliation.

Cashier: Limited to opening/closing shifts and performing sales transactions.

- Restriction on Public Registration: Only Admins have the authority to create new staff accounts.

2. Module POS Terminal (Sales & Payment)

- High-Speed Operations: Fast product searching via text box (supports barcode or product name).
- Cart Management: Add, edit quantities, and remove items from the cart.

- **Flexible Payment:** Processes payments and automatically calculates change. Supports Split Payment (e.g., partial cash and partial bank transfer).
- **Receipt Generation:** Exports transaction invoices as PDF files (simulating thermal printing) for storage and verification.

3. Module Inventory & Product Management

- **Product Lifecycle Management (CRUD):** Add, View, Update, and Delete product information (Barcode, Name, Price, Attributes).
- **Image Management:** Store and retrieve product images via Cloud Storage or local offline storage.

4. Module Shift Management

- **Open Shift:** Requires the cashier to confirm the opening cash balance in the till before selling.
- **Close Shift:** The cashier enters the actual counted amount at the end of the shift.

2.3. Non-Functional Requirements

1. **Performance:** Database optimized with SQL Indexes to ensure fast product querying and invoice generation.

2. **Security:** Passwords are hashed one-way by Cloud Auth. Implements Row Level Security (RLS) to secure the database.

3. **Usability:** Interface uses standard technical English with a minimalist design, focusing on Data Grids instead of large graphical buttons.

4. **Maintainability:** Source code architecture is clearly separated into modules: UI Logic, Database Access, and Business Logic.

2.4. Use Case Diagram

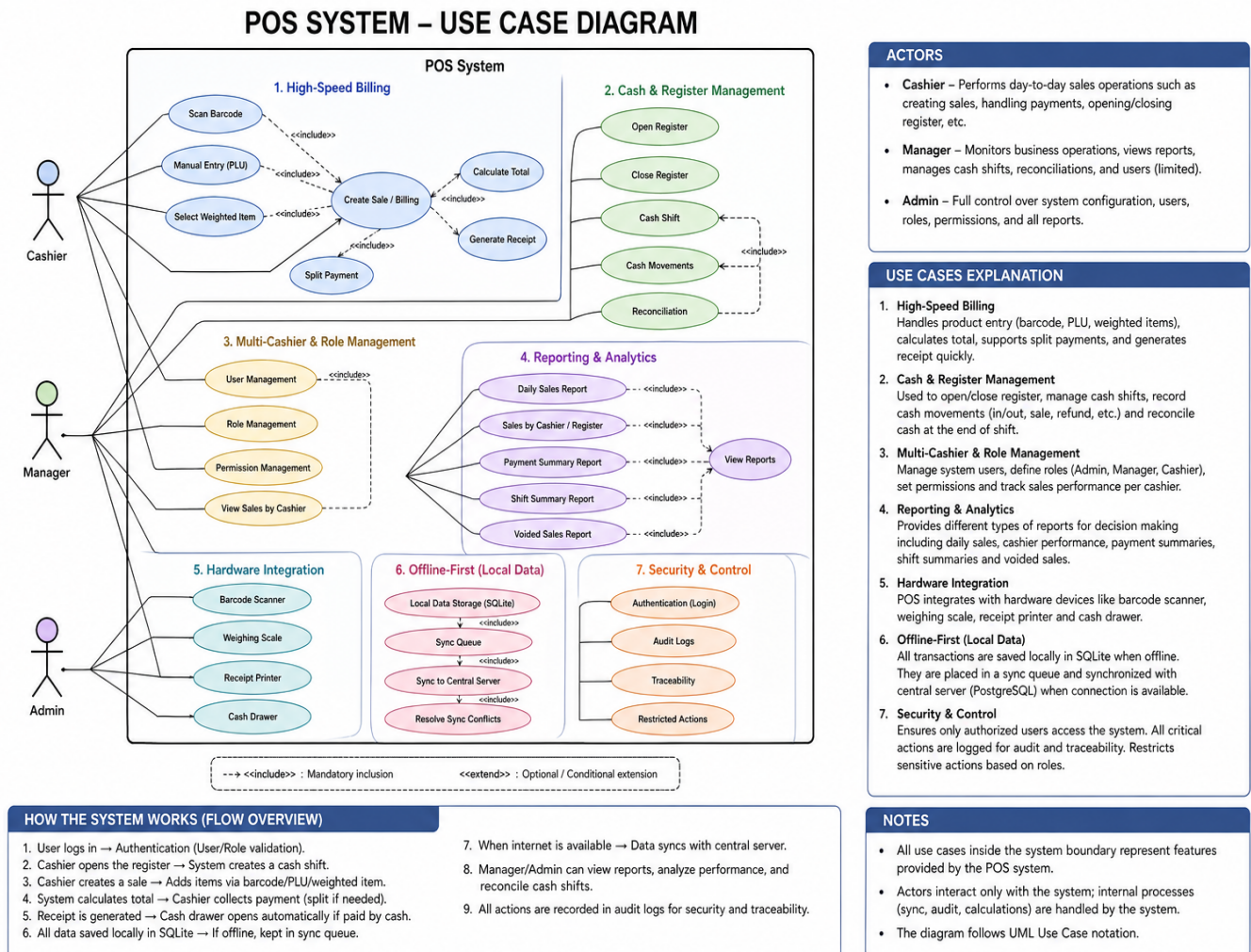


Figure 1

2.4.1 Explanation

The diagram represents a Point of Sale (POS) system where three main actors **Cashier**, **Manager**, and **Admin** interact with the system based on their responsibilities.

- The **Cashier** handles day-to-day operations like creating sales, processing payments, and managing the register during a shift.
- The **Manager** oversees operations, monitors performance, and reviews reports and cash activities.
- The **admin** controls the system at a higher level, including managing users, permissions, and overall system configuration.

All interactions happen inside the **POS** system boundary, which groups core business processes such as sales handling, cash/register control, reporting, user management, hardware usage, offline operation, and security.

The diagram shows how:

- Complex actions (like completing a sale or generating reports) are made up of smaller required steps (<<**include**>> relationships).
- The system supports real-world retail workflows, including handling cash, tracking user actions, and integrating with hardware.
- It is designed to be offline-first, meaning operations continue locally and sync later with a central system.
- Security and audit mechanisms ensure traceability and controlled access.

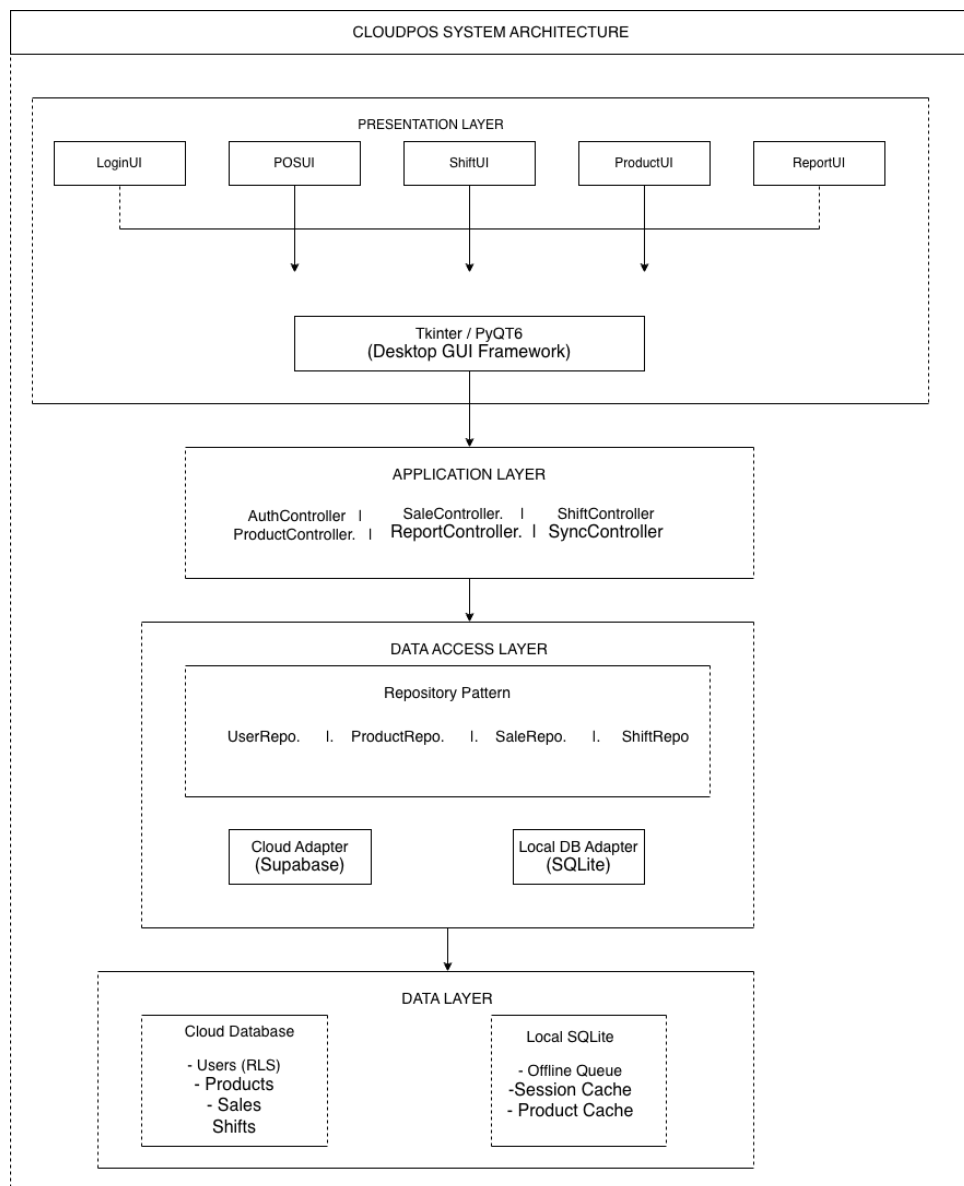
Chapter Three

3. System And Software Design

3.1. POS System – CloudPOS - System Architecture & Design Patterns

3.1.1. High-Level System Architecture

The system follows a Layered (N-Tier) Architecture combined with an Offline-First Cloud Sync Pattern. This ensures separation of concerns, maintainability, and resilience to network failures.

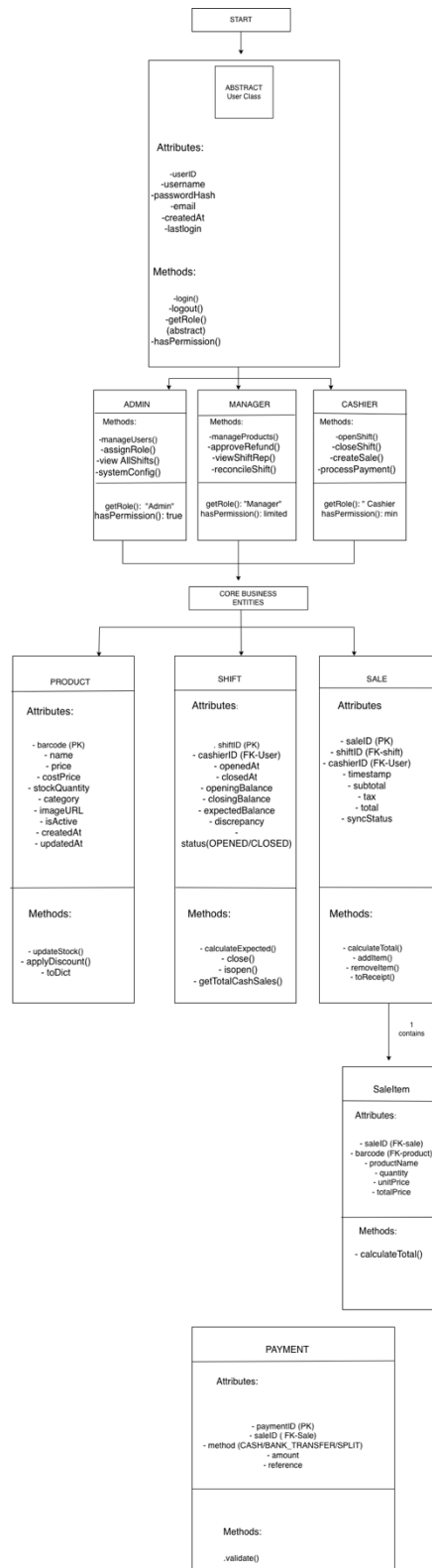


3.1.2. Design Patterns Used

Pattern	Purpose	Location in System
Repository Pattern	Abstracts data access logic, allows switching between cloud/local sources	data_access/repositories/
Singleton Pattern	Ensures single instance of database connections and sync manager	DatabaseConnection, SyncManager
Factory Pattern	Creates appropriate repository based on connectivity status	RepositoryFactory
Observer Pattern	UI updates when cart changes or sync completes	EventBus – cart update listeners, sync status listeners
Strategy Pattern	Different payment methods (cash, bank transfer, split)	PaymentStrategy interface
Command Pattern	Queues offline transactions for later execution	OfflineCommand (SaleCommand, ProductUpdateCommand)
Decorator Pattern	Adds receipt formatting options (header, footer, discounts)	ReceiptDecorator
MVC Pattern	Separates UI, logic, and data	Overall architecture

3.2. Object-Oriented Class Diagram

3.2.1. Complete UML Class Diagram



3.2.2. Offline Architecture



3.2.3. Explanation

Abstract Class Diagram

The Abstract Class Diagram illustrates the core object-oriented structure of the CloudPOS system. At the center is the abstract User class, which cannot be instantiated directly but serves as a blueprint for three specific user roles: Admin, Manager, and Cashier. The abstract class contains common attributes such as `userID`, `username`, `passwordHash`, and `email`, along with concrete methods like `login()` and `logout()` that work identically for all users. Two abstract methods, `getRole()` and `hasPermission()`, force each subclass to define its own role name and permission boundaries.

The Admin has full system access with methods to manage users and assign roles. The Manager has limited access focused on product management and shift reconciliation. The Cashier has minimal access restricted to opening shifts, creating sales, and processing payments. This three-tier hierarchy enforces strict role-based access control.

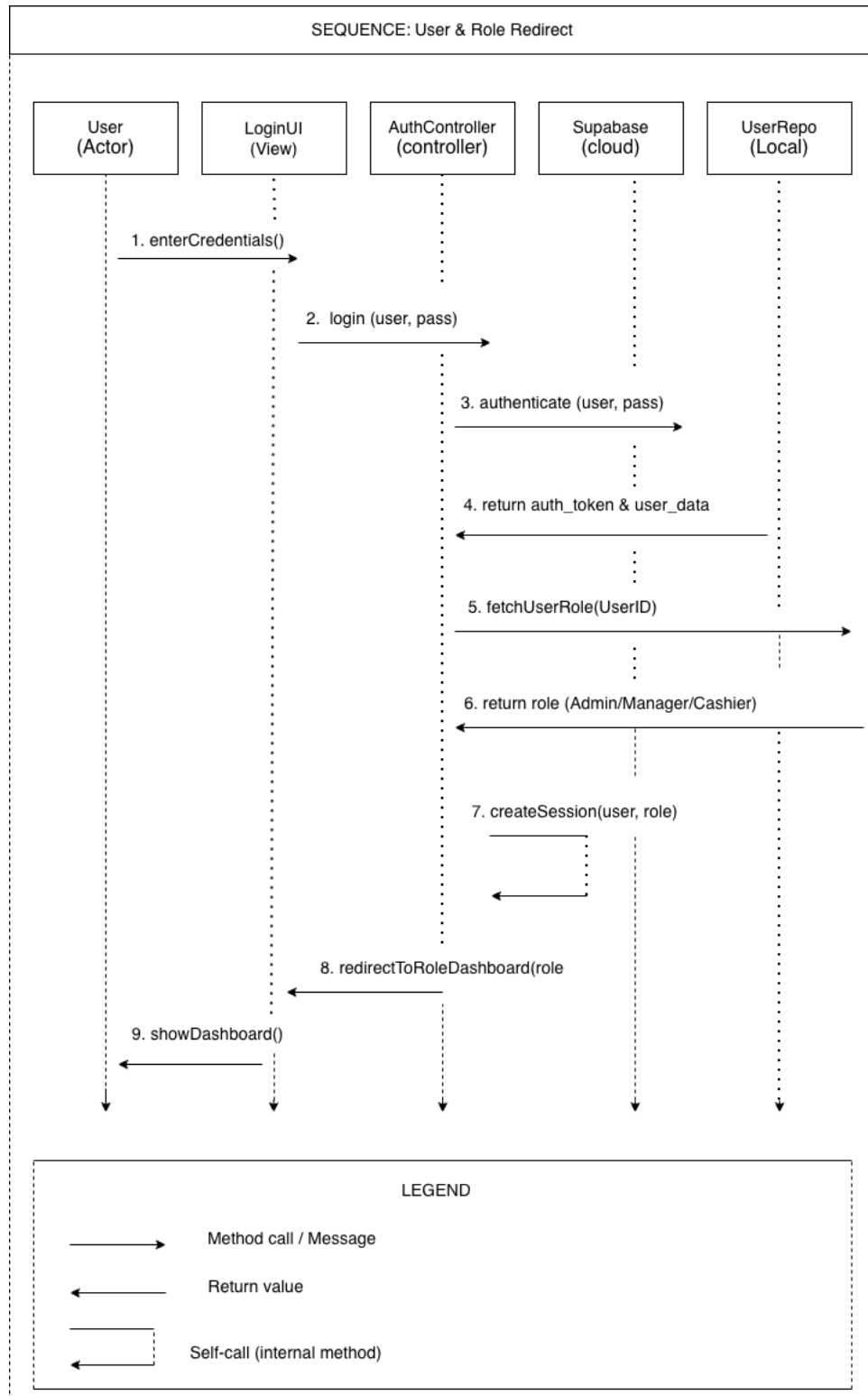
The diagram also shows four core business entities. Product stores item details including barcode, name, price, and stock quantity. Shift tracks each cashier's work session with opening balance, closing balance, expected balance, and discrepancy calculation—critical for preventing revenue leakage. Sale represents a customer transaction linked to a specific shift and cashier. SaleItem records individual line items within a sale, storing denormalized product names to preserve historical accuracy even if product information changes later.

Offline Command Diagram

The Offline Command Diagram shows how the system maintains functionality during network outages. At the top is the abstract `OfflineCommand` class, which defines attributes like `commandID`, `type`, `createdAt`, and `retryCount`, along with abstract methods `execute()` and `serialize()`. Three concrete command classes inherit from it: `SaleCommand`, `ProductUpdateCommand`, and `ShiftCommand`. Each stores the relevant data as a dictionary and implements the `execute` method to upload to the cloud when online.

The `SyncManager` uses the Singleton pattern, ensuring only one instance exists. It maintains a queue of pending commands, monitors network connectivity through the `isOnline` flag, and provides methods to enqueue commands and trigger synchronization. When the system is online, commands execute immediately on the cloud database. When offline, commands are serialized to JSON and stored in a local SQLite queue. Upon reconnection, the `SyncManager` flushes the queue in FIFO order, with failed commands retrying up to three times. This architecture ensures business continuity during internet outages while maintaining data integrity.

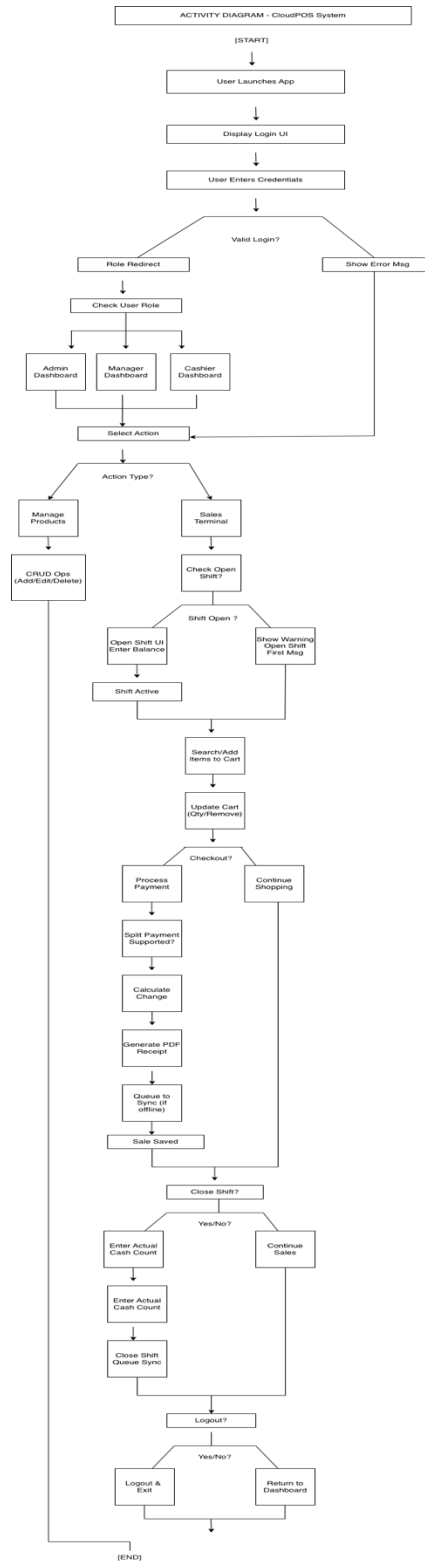
3.3. Sequence Diagram



3.3.1. Key Relationships Summary

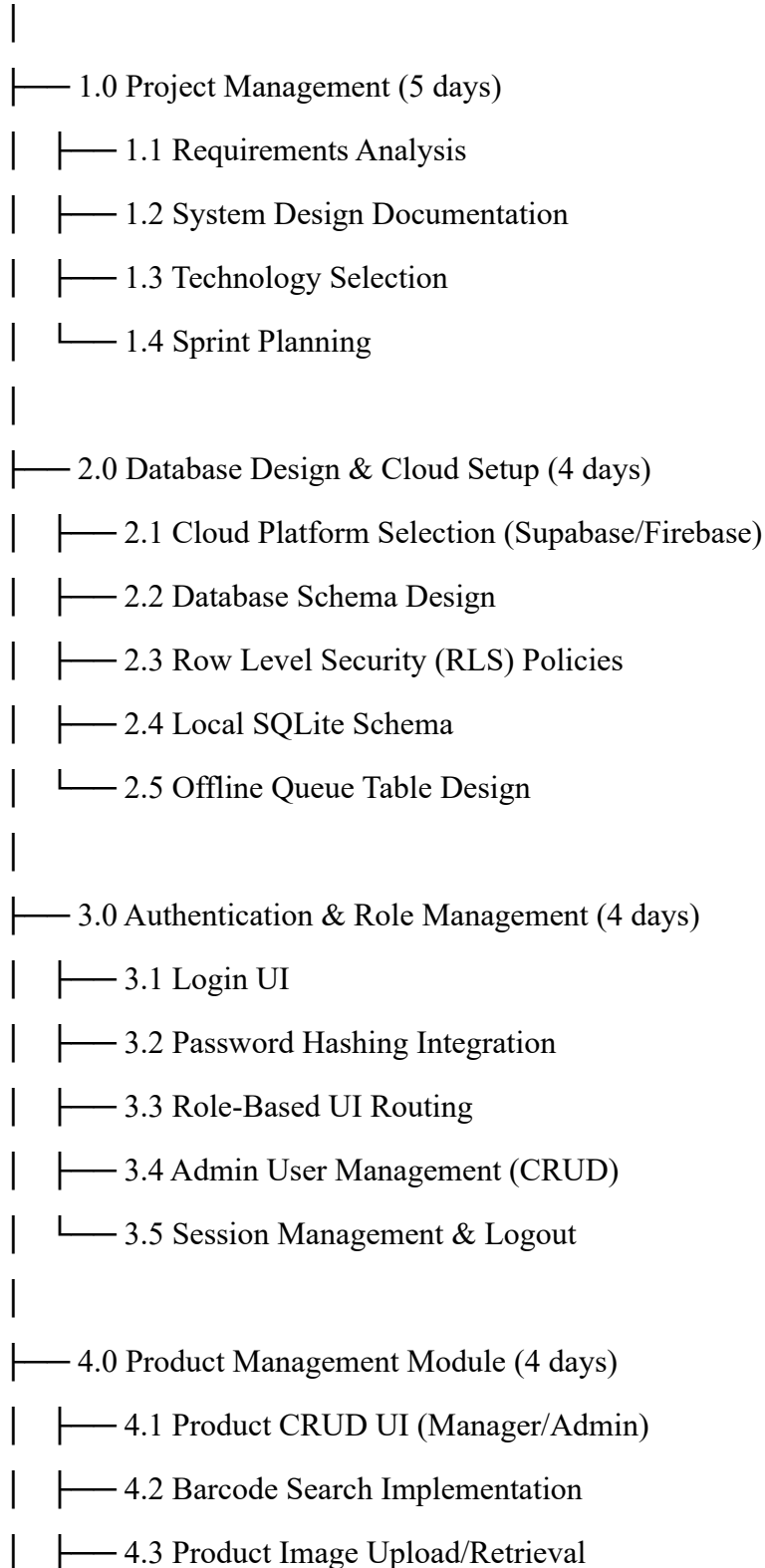
Relationship	From	To	Cardinality
Inheritance	Admin, Manager, Cashier	User	-
Composition	Sale	SaleItem	1 : *
Aggregation	Shift	Sale	1 : *
Association	Sale	Payment	1 : *
Dependency	ShiftController	Shift	-
Realization	SaleCommand	OfflineCommand	-

3.4. Activity Diagram



3.5. Work Breakdown Structure (WBS)

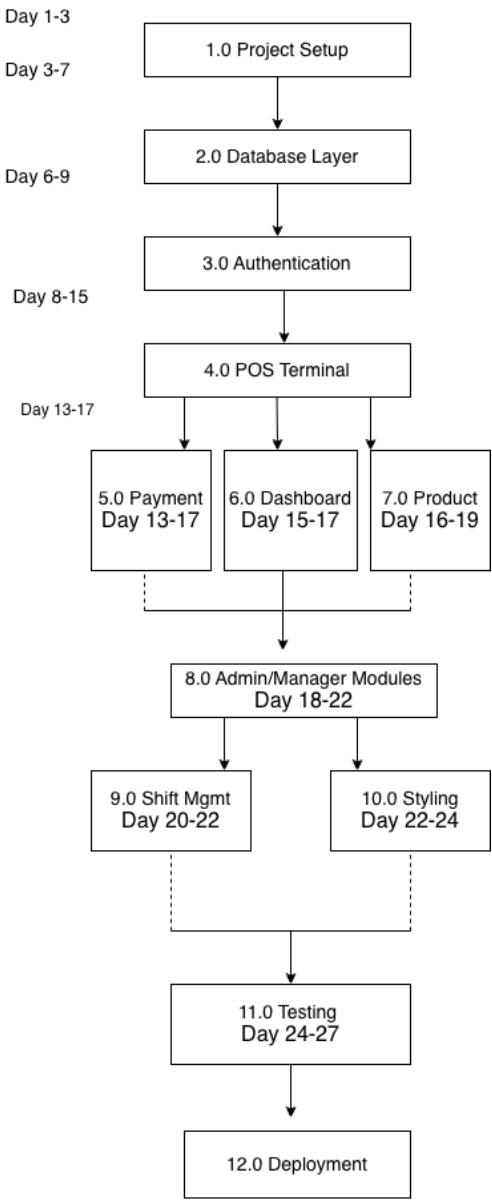
CloudPOS Project



- | |—— 4.4 Stock Quantity Tracking
- | |—— 4.5 Database Indexing for Fast Search
- |
- |—— 5.0 POS Terminal Module (6 days)
- | |—— 5.1 Cart UI (Add/Edit/Remove)
- | |—— 5.2 Fast Product Search (Text/Barcode)
- | |—— 5.3 Subtotal & Tax Calculation
- | |—— 5.4 Payment Processing UI
- | |—— 5.5 Split Payment Logic
- | |—— 5.6 Change Calculation
- | |—— 5.7 PDF Receipt Generation
- |
- |—— 6.0 Shift Management Module (4 days)
- | |—— 6.1 Open Shift UI (Opening Balance)
- | |—— 6.2 Close Shift UI (Actual Count)
- | |—— 6.3 Expected Balance Calculation
- | |—— 6.4 Discrepancy Reporting
- | |—— 6.5 Shift Status Tracking
- |
- |—— 7.0 Offline-First Sync Module (5 days)
- | |—— 7.1 Offline Command Queue
- | |—— 7.2 Network Connectivity Detection
- | |—— 7.3 SyncManager Implementation
- | |—— 7.4 Conflict Resolution Logic
- | |—— 7.5 Sync Status UI Indicator
- |
- |—— 8.0 Reporting Module (3 days)

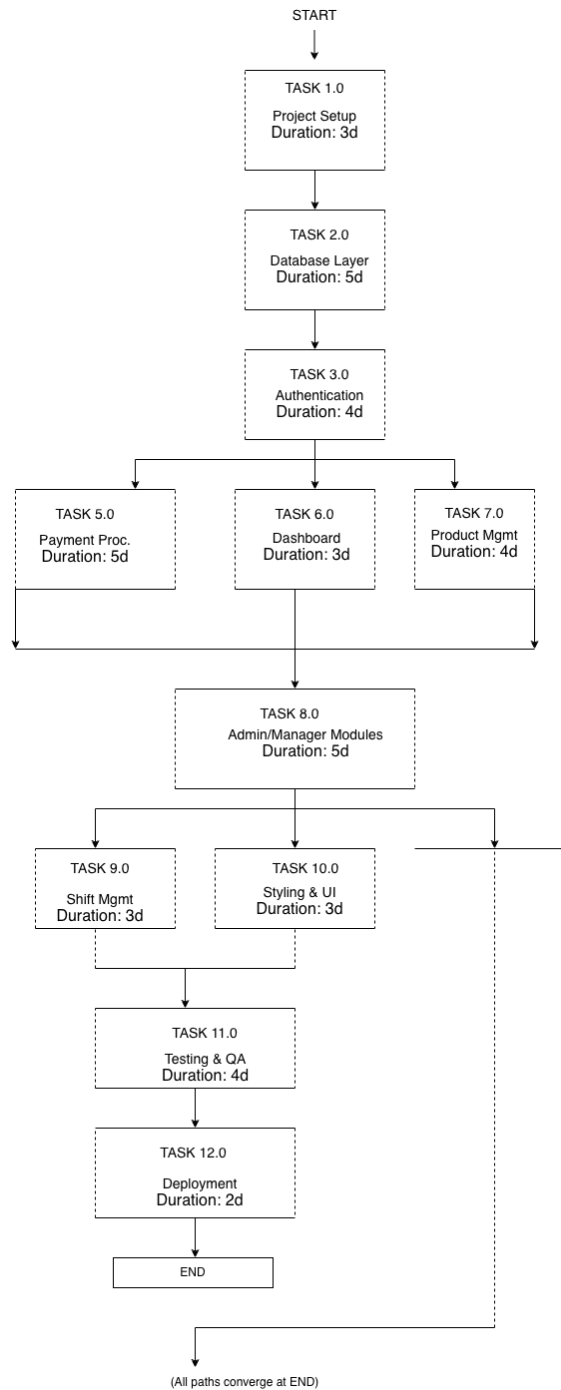
- | |—— 8.1 Shift Summary Report
- | |—— 8.2 Sales by Date/Cashier
- | |—— 8.3 Top Products Report
- | |—— 8.4 Export to CSV/PDF
- |
- |—— 9.0 Monitoring & Audit (3 days)
- | |—— 9.1 Audit Logging
- | |—— 9.2 Low Stock Alerts
- | |—— 9.3 Discrepancy Warnings
- | |—— 9.4 Sync Health Dashboard
- |
- |—— 10.0 Testing & QA (5 days)
- | |—— 10.1 Unit Testing (Controllers)
- | |—— 10.2 Integration Testing (Cloud Sync)
- | |—— 10.3 Offline Mode Testing
- | |—— 10.4 Role Permission Testing
- | |—— 10.5 UAT & Bug Fixing
- |
- |—— 11.0 Deployment & Documentation (3 days)
- | |—— 11.1 Executable Build (PyInstaller)
- | |—— 11.2 User Manual
- | |—— 11.3 Technical Documentation
- | |—— 11.4 Final Presentation Preparation

3.6. Gantt Chart



Total: 29 Days (6 weeks)

3.7. Activity Network Diagram



3.7.1. Critical Path Analysis (PERT/CPM)

Task	Duration	Predecessors	ES	EF
1.0 Setup	3	-	0	3
2.0 Database	5	1.0	3	8
3.0 Authentication	4	2.0	8	12
4.0 POS Terminal	8	2.0, 3.0	12	20
5.0 Payment	5	4.0	20	25
6.0 Dashboard	3	4.0	20	23
7.0 Product Mgmt	4	4.0	20	24
8.0 Admin Modules	5	7.0	24	29
9.0 Shift Mgmt	3	5.0	25	28
10.0 Styling	3	8.0	29	32
11.0 Testing	4	10.0	32	36
12.0 Deployment	2	11.0	36	38

Project Completion = 38 days

The POS Terminal module (Task 4.0) is the most critical task because:

- It takes 8 days (longest single task)
- Three other tasks (5.0, 6.0, 7.0) depend on it
- Any delay here directly extends the project by the same amount

Chapter Four

4. Implementation

4.1. Tools and Technologies

4.1.1. Programming Language

Technology	Version	Purpose
Python	3.10+	Core programming language for desktop application development

Why Python?

- Rapid development and prototyping capabilities
- Extensive library support for GUI, database, PDF generation, and cloud integration
- Cross-platform compatibility (Windows, Linux, macOS)
- Strong community support for POS system development

4.1.2. Graphical User Interface (GUI) Framework

Technology	Version	Purpose
Tkinter	Built-in Python 3	Primary UI framework for desktop application
ttk	Built-in	Themed widgets for modern appearance
Pillow (PIL)	9.0+	Image handling and product image display

Alternative Considered: PyQt6 (more advanced but heavier)

Selected: Tkinter – Lightweight, no additional dependencies, sufficient for minimalist data-grid design.

4.1.3. Database Technologies

Technology	Version	Purpose
Supabase	Latest	Cloud database with built-in authentication and Row Level Security (RLS)
PostgreSQL	14+	Underlying cloud database (provided by Supabase)
SQLite	3.x	Local offline database for queue storage and caching

Why Supabase?

- Built-in authentication with password hashing
- Row Level Security (RLS) for role-based data access
- Real-time capabilities for sync status
- PostgreSQL compatibility (indexes, complex queries)

Why SQLite?

- Zero-configuration local storage
- ACID compliance for transaction safety
- Perfect for offline command queue

4.1.4. Cloud & Synchronization

Technology	Version	Purpose
Supabase Client	2.0+	Python client for cloud database operations
Requests	2.28+	HTTP requests for cloud API calls
Python-dotenv	1.0+	Environment variable management for credentials

4.1.5. PDF Generation (Receipts)

Technology	Version	Purpose
ReportLab	3.6+	PDF generation for transaction receipts
FPDF	1.7+	Alternative lightweight PDF generator

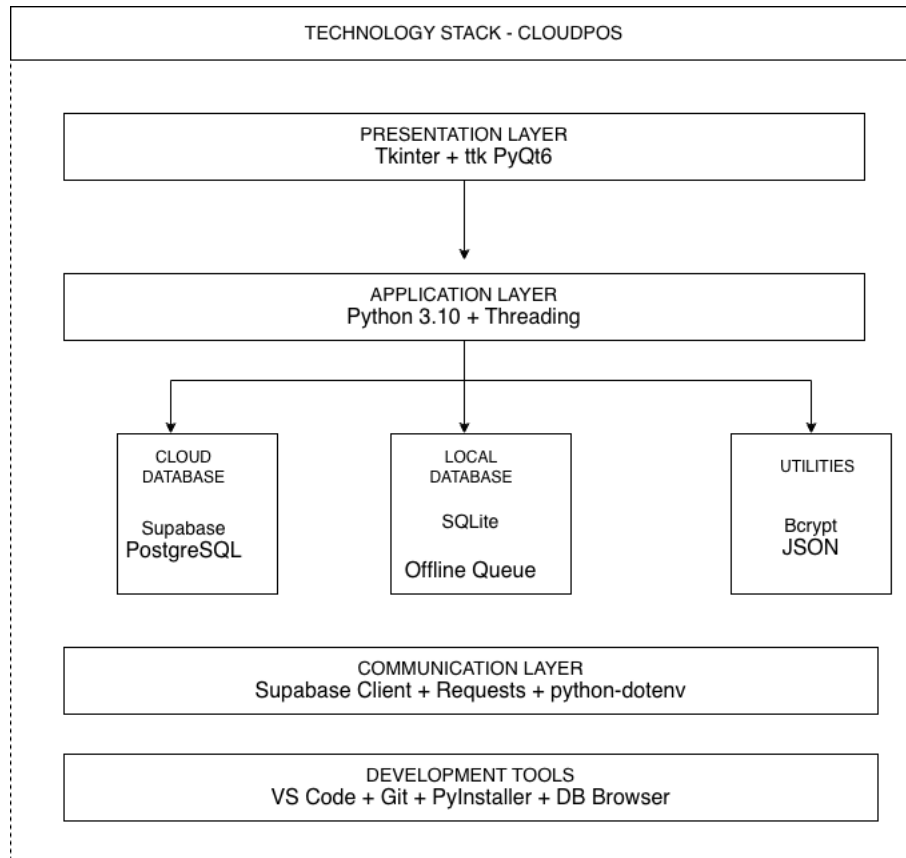
4.1.6. Development Tools

Tool	Version	Purpose
VS Code	Latest	Integrated development environment
Git	2.40+	Version control
GitHub	-	Remote repository and collaboration
Postman	Latest	API testing for cloud endpoints
DB Browser for SQLite	3.12+	Local database inspection
<u>Draw.io</u>	-	Diagram creation (architecture, class diagrams)
PyInstaller	5.0+	Executable build for distribution

4.1.7. Hardware Requirements (Deployment)

Component	Minimum Specification
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB free space
Display	1366×768 resolution
Network	Wi-Fi/Ethernet (intermittent connectivity acceptable)
OS	Windows 10/11, macOS 11+

4.2. Technology Stack Summary Diagram



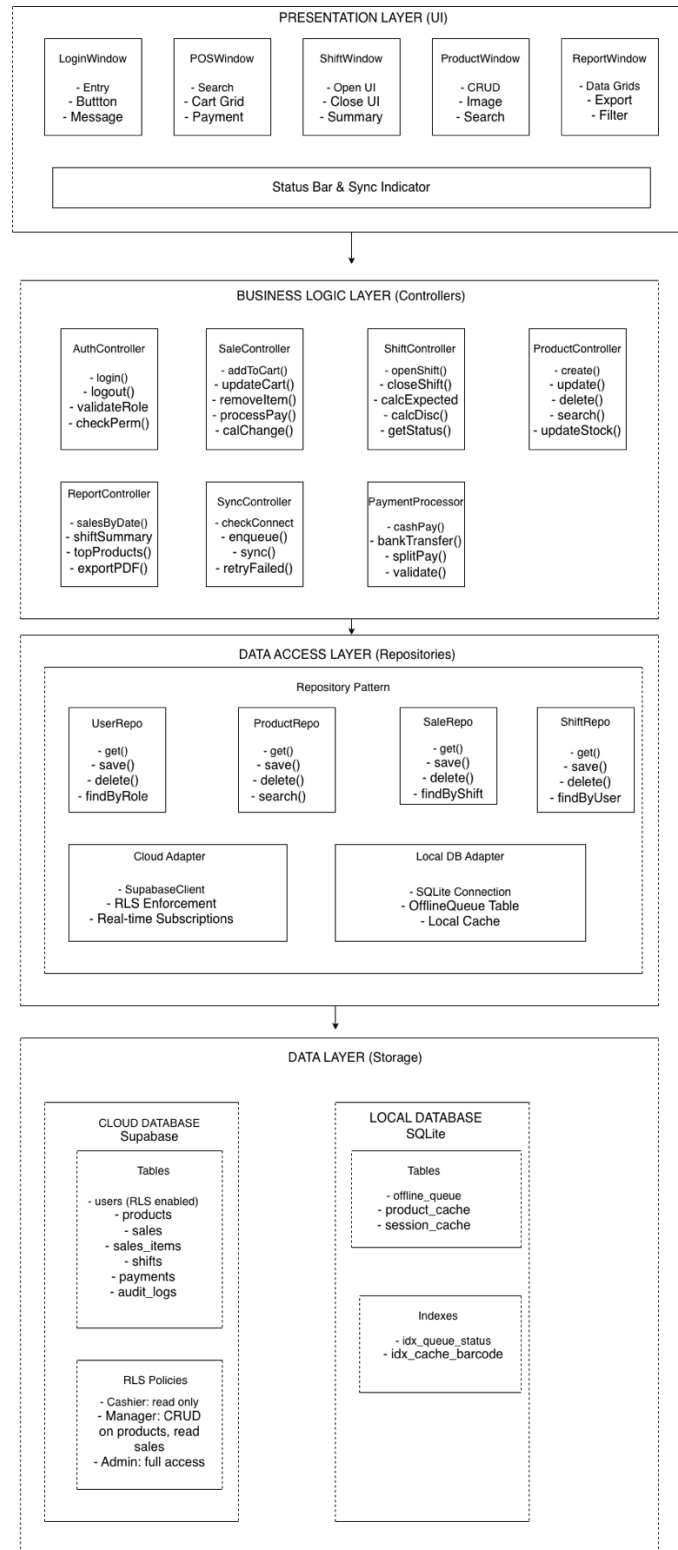
4.3. System Architecture

4.3.1. High-Level Architecture Overview

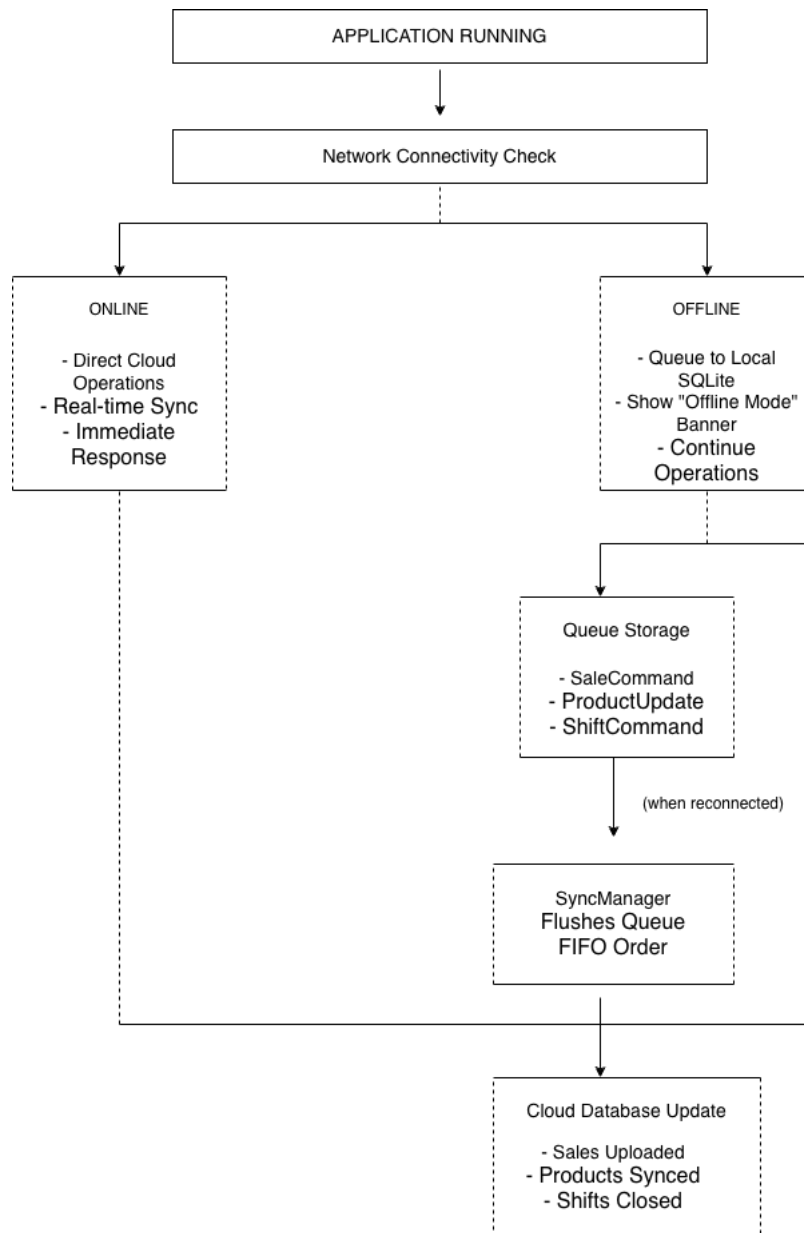
The CloudPOS system follows a Layered (N-Tier) Architecture with Offline-First capabilities. The architecture is divided into four distinct layers:

Layer	Responsibility	Components
Presentation Layer	User interface and user interaction	Tkinter windows, dialogs, data grids
Business Logic Layer	Core application logic and rules	Controllers, validators, calculators
Data Access Layer	Database abstraction and synchronization	Repositories, SyncManager, Cloud/Local adapters
Data Layer	Persistent storage	Supabase Cloud, SQLite Local

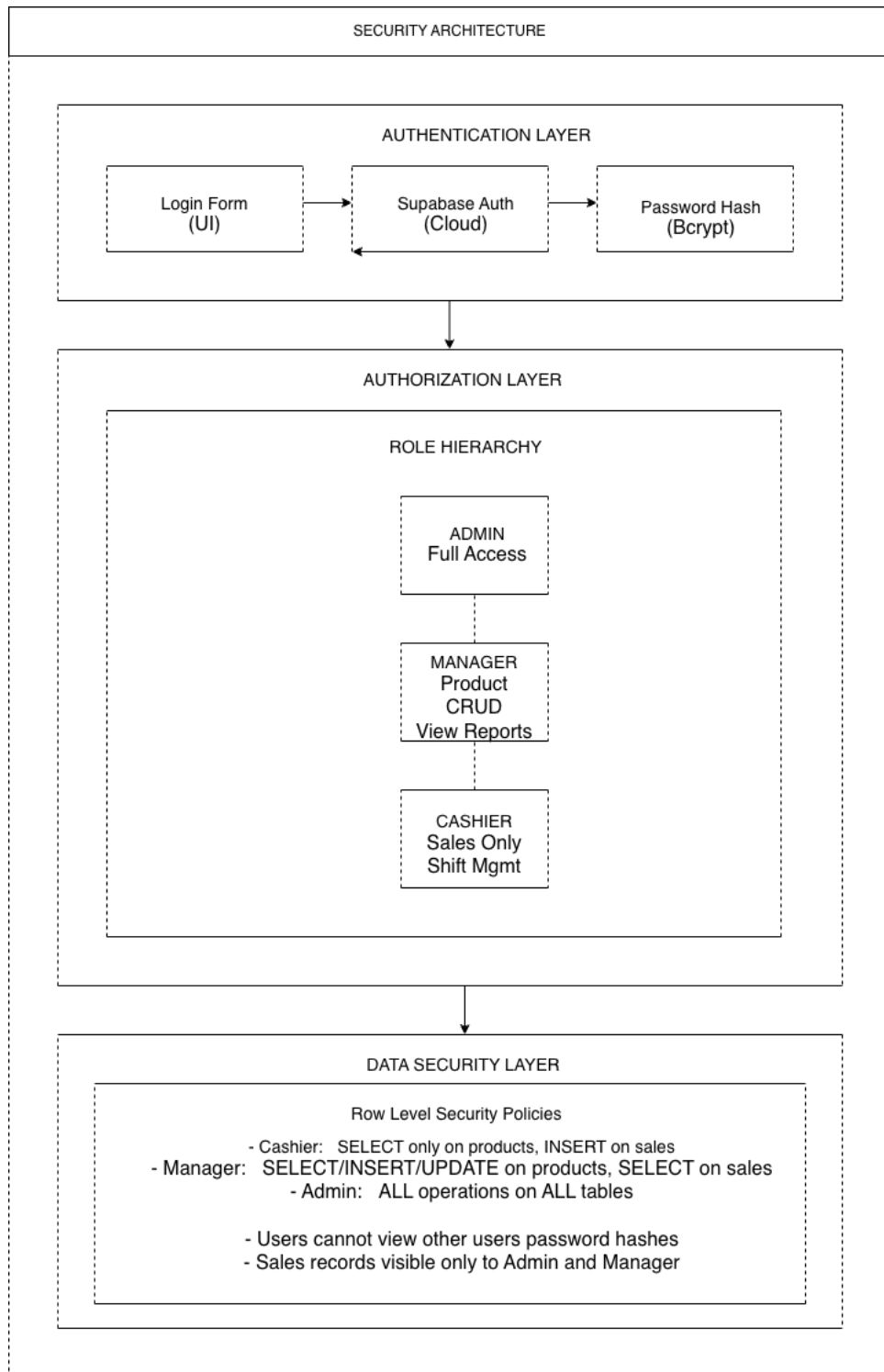
4.3.2. Architecture Diagram



4.3.3. Offline-First Sync Architecture



4.3.4. Security Architecture



4.4. System Modules

The system is divided into **seven core modules**, each responsible for a specific functional area.

Module 1: Authentication & Role Management

Purpose: Secure user login and role-based access enforcement.

Components:

- Login UI with credential validation
- Password hashing (Supabase Auth)
- Session management with timeout
- Role-based dashboard routing
- Admin-only user management (CRUD)

Key Files:

/modules/auth/

```
|— login_window.py
|— auth_controller.py
|— session_manager.py
|— role_decorator.py
```

Module 2: POS Terminal (Sales & Payment)

Purpose: High-speed checkout operations with cart management and split payments.

Components:

- Product search (barcode/name) with auto-suggest
- Shopping cart UI (add, edit quantity, remove)
- Subtotal, tax, total calculation
- Payment processing (cash, bank transfer, split)
- Change calculation
- PDF receipt generation

Key Files:

/modules/pos/

```
|— pos_window.py
|— cart_manager.py
|— payment_processor.py
|— receipt_generator.py
└— barcode_scanner.py
```

Module 3: Inventory & Product Management

Purpose: Complete product lifecycle management with image support.

Components:

- Product CRUD operations (Create, Read, Update, Delete)
- Barcode-based product search
- Product image upload and retrieval (cloud/local)
- Stock quantity tracking
- Category management
- Database indexing for fast search

Key Files:

/modules/inventory/

```
|— product_window.py
|— product_controller.py
|— product_repository.py
|— image_handler.py
└— barcode_generator.py
```

Module 4: Shift Management

Purpose: Track cashier work sessions and prevent revenue leakage.

Components:

- Open shift UI (opening balance entry)
- Close shift UI (actual cash count)
- Expected balance calculation
- Discrepancy detection and reporting
- Shift status tracking (OPEN/CLOSED)
- Shift history viewer

Key Files:

/modules/shift/

```
|— shift_window.py  
|— shift_controller.py  
|— shift_repository.py  
└— shift_calculator.py
```

Module 5: Offline-First Synchronization

Purpose: Ensure business continuity during network outages.

Components:

- Network connectivity monitor
- Offline command queue (SQLite)
- SyncManager (Singleton pattern)
- Conflict resolution strategy
- Sync status UI indicator
- Retry mechanism for failed commands

Key Files:

/modules/sync/

- |— sync_manager.py
- |— offline_command.py
- |— sale_command.py
- |— product_update_command.py
- |— shift_command.py
- |— network_monitor.py
- |— conflict_resolver.py

Module 6: Reporting & Analytics

Purpose: Provide business insights and operational visibility.

Components:

- Shift summary report
- Sales by date/cashier report
- Top products report
- Discrepancy report
- Export to PDF/CSV
- Data grid visualizations

Key Files:

/modules/reporting/

- |— report_window.py
- |— report_controller.py
- |— report_generator.py
- |— export_handler.py
- |— chart_builder.py

Report Types:

Report Name	Data Shown	Target User
Shift Summary	Opening/closing balances, discrepancy, transaction count	Manager, Admin
Daily Sales	Total revenue, transactions, average ticket	Manager, Admin
Cashier Performance	Sales per cashier, average shift duration	Admin
Top Products	Best-selling items by quantity and revenue	Manager
Discrepancy Report	Shifts with positive/negative variance	Admin

Module 7: Audit & Monitoring

Purpose: Track all system actions and maintain security.

Components:

- Audit logger (all user actions)
- Login attempt tracking
- Shift discrepancy alerts
- Low stock notifications
- Sync health dashboar

Key Files:

/modules/monitoring/

- |— audit_logger.py
- |— alert_manager.py
- |— health_checker.py
- |— dashboard_window.py

4.5. Key Features

4.5.1. Feature Summary Table

#	Feature	Description	Implemented In
1	Role-Based Access Control	3-tier hierarchy (Admin, Manager, Cashier) with strict permissions	Module 1
2	Secure Authentication	Cloud-based password hashing, no plain-text storage	Module 1
3	Fast Product Search	Barcode/name search with database indexing (<500ms response)	Module 2
4	Shopping Cart	Add, edit quantities, remove items in real-time	Module 2
5	Split Payment	Partial cash + partial bank transfer in single transaction	Module 2
6	PDF Receipts	Automatic receipt generation after each sale	Module 2

#	Feature	Description	Implemented In
7	Product Management	Full CRUD operations with image support	Module 3
8	Shift Opening	Cashier confirms opening balance before sales	Module 4
9	Shift Closing	Actual cash count vs. expected balance with discrepancy	Module 4
10	Offline-First Sync	Continue sales offline, auto-sync when reconnected	Module 5
11	Conflict Resolution	Cloud-wins for products, local-wins for sales	Module 5
12	Sync Status Indicator	Real-time connectivity and sync status in UI	Module 5
13	Sales Reports	Daily, shift, and cashier performance reports	Module 6
14	Audit Trail	Complete log of all user actions	Module 7
15	Low Stock Alerts	Automatic notification when stock < threshold	Module 7

4.6. Screenshots

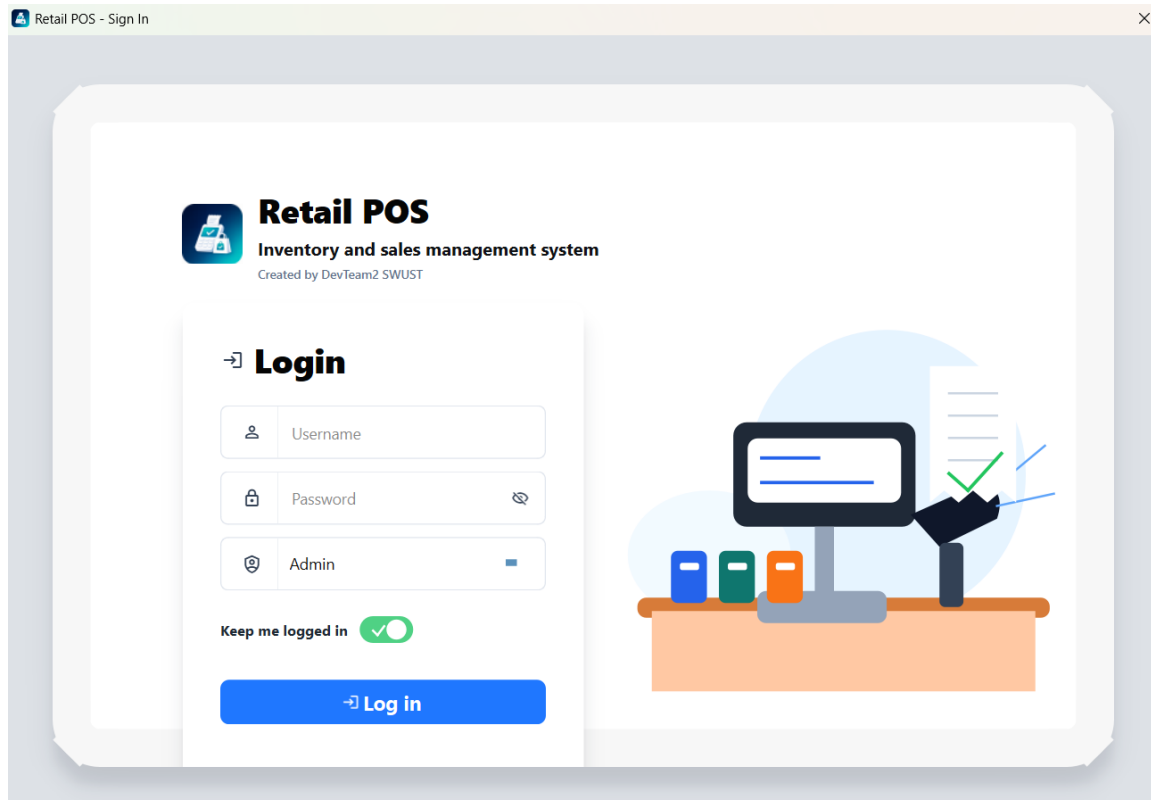


Figure 2. Login Page

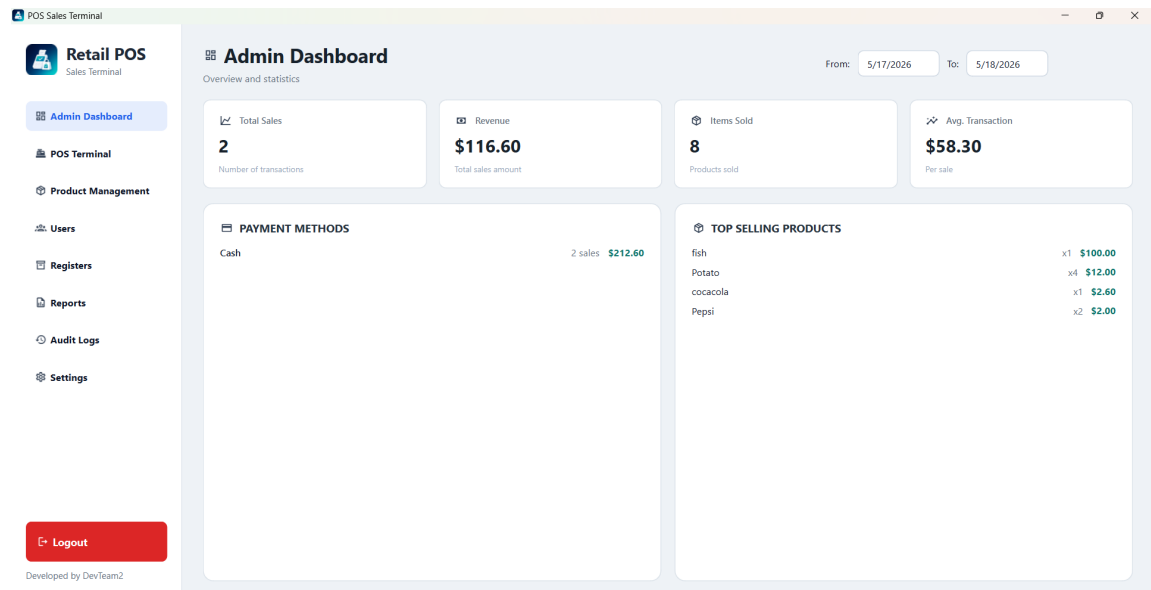


Figure 3. Admin's Dashboard

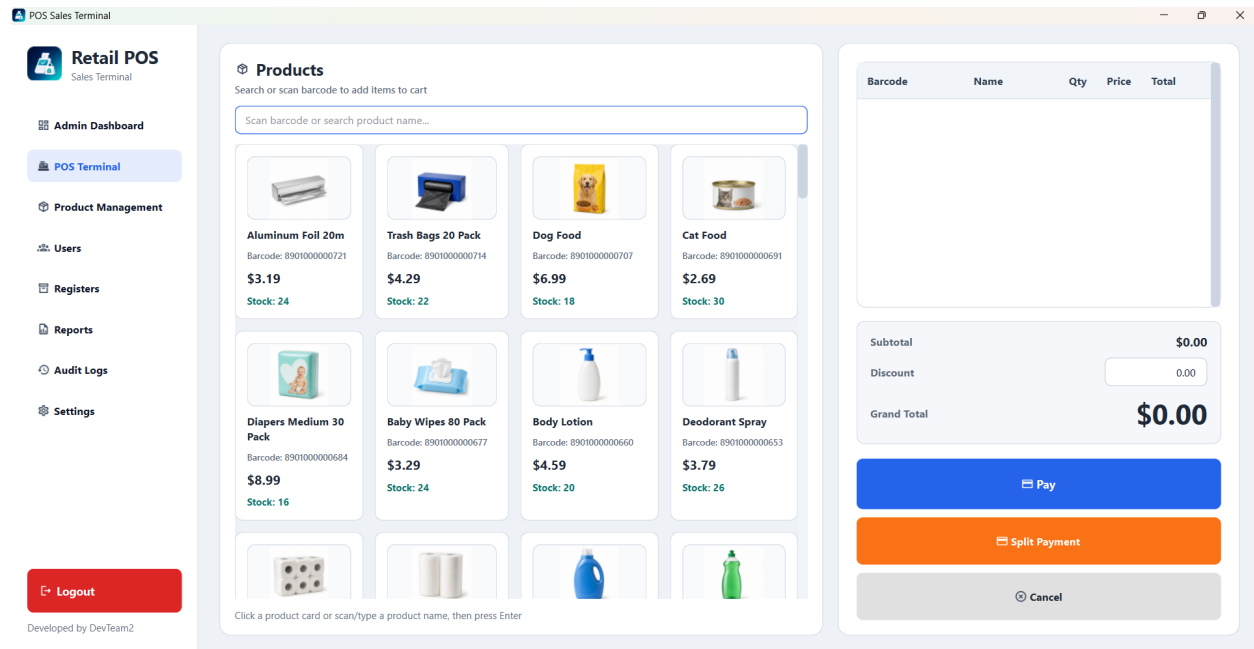


Figure 4. Products

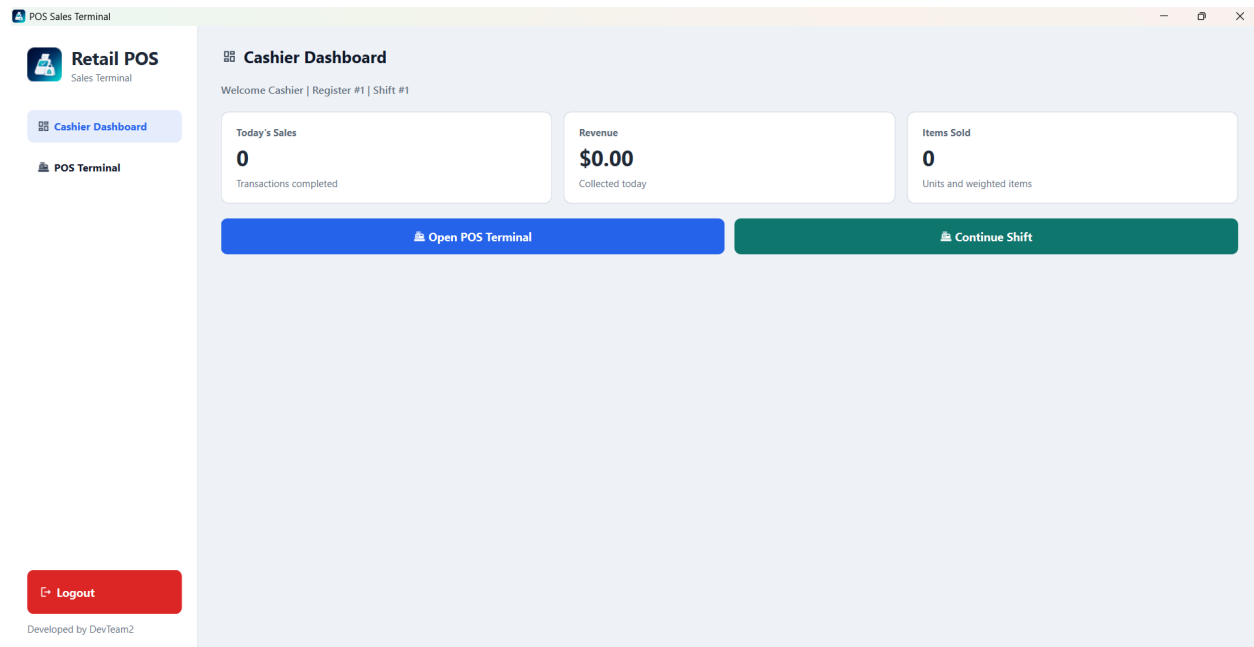


Figure 5. Cashier's Dashboard

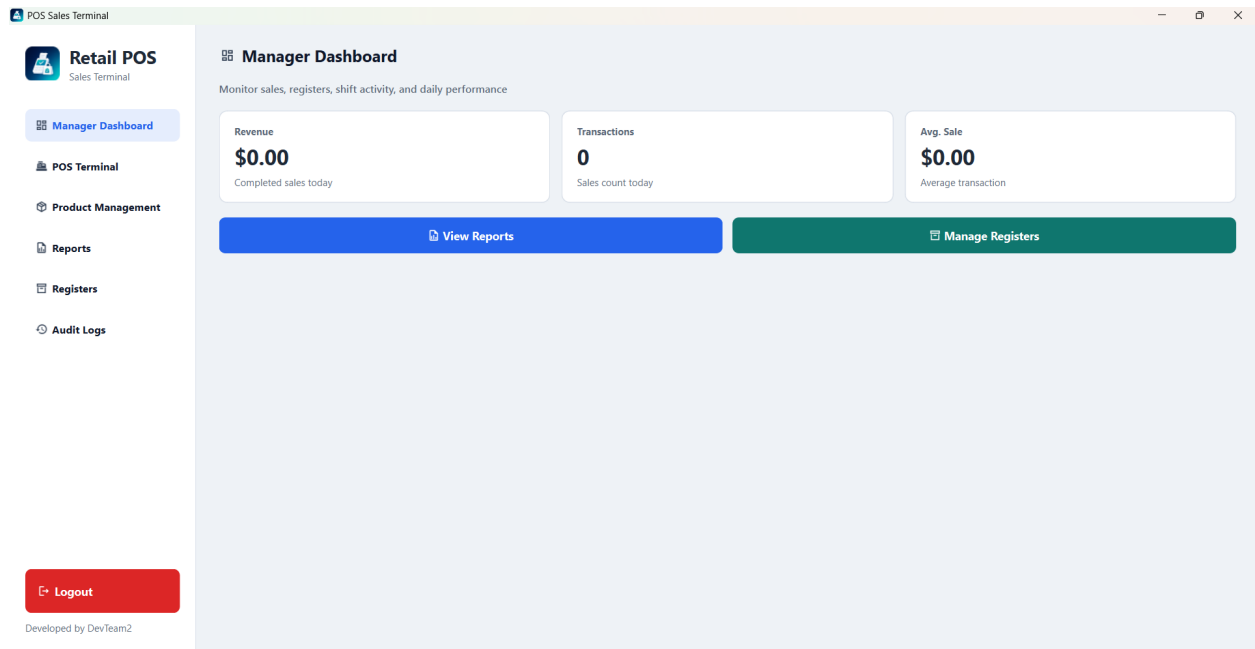


Figure 6. Manager's Dashboard